Rust libp2p Relay 서버 배포 및 운영 가이드

개요

이 가이드는 Rust libp2p relay 서버를 프로덕션 환경에 배포하고 운영하는 방법을 상세히 설명합니다. NAT traversal, 네트워크 연결성 향상, 그리고 분산 네트워크의 안정성을 위한 relay 서버 구축 방법을 다룹니다.

배포 아키텍처

단일 Relay 서버

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│   Client A   │────│    Relay     │────│   Client B   │
│   (NAT 뒤)   │    │    Server    │    │   (NAT 뒤)   │
└──────────────┘    └──────────────┘    └──────────────┘
```

다중 Relay 서버 (고가용성)

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│   Client A   │────│   Relay 1    │────│   Client B   │
│              │    └──────────────┘    │              │
│              │    ┌──────────────┐    │              │
│              │────│   Relay 2    │────│              │
```

```
│              │    └──────────────┘    │              │

│              │    ┌──────────────┐    │              │

│              │────│   Relay 3    │────│              │

└──────────────┘    └──────────────┘    └──────────────┘
```

서버 설정 및 구성

1. 기본 Relay 서버 설정

```rust
use libp2p::{relay, identity, noise, tcp, yamux, ping, identify};

use std::{time::Duration, num::NonZeroU32};


// 프로덕션용 relay 설정

let relay_config = relay::Config {

// 예약 관련 설정

max_reservations: 1024, // 대규모 네트워크 지원

max_reservations_per_peer: 8, // 피어당 예약 수 증가

reservation_duration: Duration::from_secs(7200), // 2시간

// 회로 관련 설정

max_circuits: 256, // 동시 회로 수 증가

max_circuits_per_peer: 8, // 피어당 회로 수 증가

max_circuit_duration: Duration::from_secs(600), // 10분

max_circuit_bytes: 10 << 20, // 10MB
```

```
..Default::default()

}

// Rate limiting 설정 (DDoS 방지)

.reservation_rate_per_peer(

NonZeroU32::new(60).unwrap(), // 시간당 60회

Duration::from_secs(60) // 1분 간격

)

.reservation_rate_per_ip(

NonZeroU32::new(120).unwrap(), // IP당 시간당 120회

Duration::from_secs(30) // 30초 간격

)

.circuit_src_per_peer(

NonZeroU32::new(60).unwrap(),

Duration::from_secs(60)

)

.circuit_src_per_ip(

NonZeroU32::new(120).unwrap(),

Duration::from_secs(30)

);
```

2. 네트워크 설정

```
// 다중 프로토콜 지원
```

```
let  mut swarm = libp2p::SwarmBuilder::with_existing_identity(keypair)

.with_tokio()

.with_tcp(

tcp::Config::default()

.port_reuse(true) // 포트 재사용 활성화

.nodelay(true), // TCP_NODELAY 설정

noise::Config::new,

yamux::Config::default()

.with_max_buffer_size(16 * 1024 * 1024) // 16MB 버퍼

.with_max_num_streams(1024), // 최대 스트림 수

)?

.with_quic() // QUIC 지원

.with_behaviour(|key| RelayBehaviour {

relay: relay::Behaviour::new(key.public().to_peer_id(), relay_config),

ping: ping::Behaviour::new(ping::Config::new()

.with_interval(Duration::from_secs(30))

.with_timeout(Duration::from_secs(10))

),

identify: identify::Behaviour::new(identify::Config::new(

"/relay-server/1.0.0".to_string(),

key.public(),

)),

})?
```

```
.build();


// 다중 주소 리스닝
let addresses = vec![
"/ip4/0.0.0.0/tcp/4001".parse()?, // IPv4 TCP
"/ip6/::/tcp/4001".parse()?, // IPv6 TCP
"/ip4/0.0.0.0/udp/4001/quic-v1".parse()?, // IPv4 QUIC
"/ip6/::/udp/4001/quic-v1".parse()?, // IPv6 QUIC
];


for addr in addresses {
swarm.listen_on(addr)?;
}
```

3. 보안 설정

```
// 강화된 보안 설정
let noise_config = noise::Config::new(&keypair)?
.with_prologue(b"libp2p-relay-v2") // 프로토콜 식별자
.with_xx_handshake(); // XX 핸드셰이크 패턴


// 연결 제한 설정
let connection_limits = libp2p::swarm::ConnectionLimits::default()
```

```
.with_max_pending_incoming(Some(64))
.with_max_pending_outgoing(Some(64))
.with_max_established_incoming(Some(512))
.with_max_established_outgoing(Some(512))
.with_max_established_per_peer(Some(8));

swarm = swarm.with_swarm_config(|cfg| {
cfg.with_connection_limits(connection_limits)
});
```

모니터링 및 로깅

1. 구조화된 로깅

```
use tracing::{info, warn, error, debug, span, Level};
use tracing_subscriber::{layer::SubscriberExt, util::SubscriberInitExt};

// 로깅 초기화
tracing_subscriber::registry()
.with(tracing_subscriber::EnvFilter::new("info,libp2p_relay=debug"))
.with(tracing_subscriber::fmt::layer()
.with_target(true)
.with_thread_ids(true)
```

```
.with_file(true)
.with_line_number(true)
)
.init();


// 메트릭 수집을 위한 구조체
#[derive(Default)]
struct RelayMetrics {
active_reservations: std::sync::atomic::AtomicUsize,
active_circuits: std::sync::atomic::AtomicUsize,
total_reservations: std::sync::atomic::AtomicU64,
total_circuits: std::sync::atomic::AtomicU64,
bytes_relayed: std::sync::atomic::AtomicU64,
}


// 이벤트 처리 시 메트릭 업데이트
fn handle_relay_event(event: relay::Event, metrics: &RelayMetrics) {
match event {
relay::Event::ReservationReqAccepted { src_peer_id, renewed } => {
if !renewed {
metrics.active_reservations.fetch_add(1, std::sync::atomic::Ordering::Relaxed);
}
metrics.total_reservations.fetch_add(1, std::sync::atomic::Ordering::Relaxed);
```

```
info!(
peer_id = %src_peer_id,
renewed = renewed,
active_reservations = metrics.active_reservations.load(std::sync::atomic::Orderi
"Reservation accepted"
);
}
relay::Event::CircuitReqAccepted { src_peer_id, dst_peer_id } => {
metrics.active_circuits.fetch_add(1, std::sync::atomic::Ordering::Relaxed);
metrics.total_circuits.fetch_add(1, std::sync::atomic::Ordering::Relaxed);
info!(
src_peer = %src_peer_id,
dst_peer = %dst_peer_id,
active_circuits = metrics.active_circuits.load(std::sync::atomic::Ordering::Rela
"Circuit established"
);
}
relay::Event::CircuitClosed { src_peer_id, dst_peer_id, error } => {
metrics.active_circuits.fetch_sub(1, std::sync::atomic::Ordering::Relaxed);
if  let Some(err) = error {
warn!(
src_peer = %src_peer_id,
dst_peer = %dst_peer_id,
```

```
error = %err,
"Circuit closed with error"
);
} else {
info!(
src_peer = %src_peer_id,
dst_peer = %dst_peer_id,
"Circuit closed normally"
);
}
}
_ => {}
}
}
```

2. 헬스 체크 엔드포인트

```
use warp::Filter;
use serde_json::json;

// HTTP 헬스 체크 서버
async fn start_health_server(metrics: Arc<RelayMetrics>) {
```

```
let health = warp::path("health")
.map(|| warp::reply::with_status("OK", warp::http::StatusCode::OK));
let metrics_endpoint = warp::path("metrics")
.map(move || {
let active_reservations = metrics.active_reservations.load(std::sync::atomic::Or
let active_circuits = metrics.active_circuits.load(std::sync::atomic::Ordering::
let total_reservations = metrics.total_reservations.load(std::sync::atomic::Orde
let total_circuits = metrics.total_circuits.load(std::sync::atomic::Ordering::Re
let bytes_relayed = metrics.bytes_relayed.load(std::sync::atomic::Ordering::Rela
warp::reply::json(&json!({
"active_reservations": active_reservations,
"active_circuits": active_circuits,
"total_reservations": total_reservations,
"total_circuits": total_circuits,
"bytes_relayed": bytes_relayed,
"uptime_seconds": std::time::SystemTime::now()
.duration_since(std::time::UNIX_EPOCH)
.unwrap()
.as_secs()
}))
});
let routes = health.or(metrics_endpoint);
warp::serve(routes)
```

```
.run(([0, 0, 0, 0], 8080))

.await;

}
```

Docker 배포

1. Dockerfile

```
# Multi-stage build for optimized image

FROM rust:1.75-slim as builder


WORKDIR /app

COPY . .


# Install dependencies

RUN apt-get update && apt-get install -y \

pkg-config \

libssl-dev \

&& rm -rf /var/lib/apt/lists/*


# Build the application

RUN cargo build --release
```

```
# Runtime image
FROM debian:bookworm-slim

# Install runtime dependencies
RUN apt-get update && apt-get install -y \
ca-certificates \
&& rm -rf /var/lib/apt/lists/*

# Create non-root user
RUN useradd -r -s /bin/false relay

# Copy binary
COPY --from=builder /app/target/release/relay-server /usr/local/bin/relay-server

# Set ownership
RUN chown relay:relay /usr/local/bin/relay-server

USER relay

# Expose ports
EXPOSE 4001 8080

# Health check
```

```
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
CMD curl -f http://localhost:8080/health || exit 1


CMD ["relay-server", "--port", "4001"]
```

2. Docker Compose

```
version: '3.8'


services:
relay-server:
build: .
ports:
- "4001:4001"  # P2P port
- "8080:8080"  # Health check port
environment:
- RUST_LOG=info,libp2p_relay=debug
volumes:
- relay-data:/data
restart: unless-stopped
networks:
- relay-network
deploy:
```

```yaml
resources:
limits:
memory: 512M
cpus: '0.5'
reservations:
memory: 256M
cpus: '0.25'


# Prometheus monitoring (optional)
prometheus:
image: prom/prometheus:latest
ports:
- "9090:9090"
volumes:
- ./prometheus.yml:/etc/prometheus/prometheus.yml
- prometheus-data:/prometheus
networks:
- relay-network


# Grafana dashboard (optional)
grafana:
image: grafana/grafana:latest
ports:
```

```yaml
    - "3000:3000"
    environment:
    - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
    - grafana-data:/var/lib/grafana
    networks:
    - relay-network


volumes:
  relay-data:
  prometheus-data:
  grafana-data:


networks:
  relay-network:
    driver: bridge
```

Kubernetes 배포

1. Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
```

```yaml
metadata:
name: libp2p-relay
labels:
app: libp2p-relay
spec:
replicas: 3
selector:
matchLabels:
app: libp2p-relay
template:
metadata:
labels:
app: libp2p-relay
spec:
containers:
- name: relay-server
image: libp2p-relay:latest
ports:
- containerPort: 4001
name: p2p
- containerPort: 8080
name: health
env:
```

```yaml
- name: RUST_LOG
value: "info,libp2p_relay=debug"
resources:
requests:
memory: "256Mi"
cpu: "250m"
limits:
memory: "512Mi"
cpu: "500m"
livenessProbe:
httpGet:
path: /health
port: 8080
initialDelaySeconds: 30
periodSeconds: 10
readinessProbe:
httpGet:
path: /health
port: 8080
initialDelaySeconds: 5
periodSeconds: 5
```

2. Service

```yaml
apiVersion: v1

kind: Service

metadata:

name: libp2p-relay-service

spec:

selector:

app: libp2p-relay

ports:

- name: p2p

port: 4001

targetPort: 4001

protocol: TCP

- name: health

port: 8080

targetPort: 8080

protocol: TCP

type: LoadBalancer
```

성능 튜닝

1. 시스템 레벨 최적화

```
# /etc/sysctl.conf 설정
```

```
net.core.rmem_max =  134217728

net.core.wmem_max =  134217728

net.ipv4.tcp_rmem =  4096  87380  134217728

net.ipv4.tcp_wmem =  4096  65536  134217728

net.core.netdev_max_backlog =  5000

net.ipv4.tcp_congestion_control =  bbr


# 파일 디스크립터 제한 증가

echo "* soft nofile 65536" >> /etc/security/limits.conf

echo "* hard nofile 65536" >> /etc/security/limits.conf
```

2. 애플리케이션 레벨 최적화

```
// 고성능 설정

let relay_config = relay::Config {

max_reservations: 4096,

max_reservations_per_peer: 16,

max_circuits: 1024,

max_circuits_per_peer: 16,

max_circuit_bytes: 100 << 20, // 100MB

..Default::default()

};
```

```
// Yamux 최적화
let yamux_config = yamux::Config::default()
.with_max_buffer_size(64 * 1024 * 1024) // 64MB
.with_max_num_streams(4096)
.with_window_update_mode(yamux::WindowUpdateMode::OnRead);
```

보안 고려사항

1. 방화벽 설정

```
# UFW 설정 예제
ufw allow  4001/tcp  # P2P TCP
ufw allow  4001/udp  # P2P QUIC
ufw allow  8080/tcp  # Health check (내부 네트워크만)
ufw enable
```

2. 인증서 관리

```
// TLS 인증서를 사용한 보안 강화 (선택사항)
use libp2p::tls;

let tls_config = tls::Config::new(&keypair)?;
```

운영 체크리스트

배포 전 체크리스트

- [ ] 키 관리 및 백업 완료
- [ ] 네트워크 포트 개방 확인
- [ ] 모니터링 시스템 설정
- [ ] 로그 수집 시스템 구성
- [ ] 백업 및 복구 절차 수립
- [ ] 보안 설정 검토

운영 중 모니터링 항목

- [ ] 활성 예약 수
- [ ] 활성 회로 수
- [ ] CPU 및 메모리 사용률
- [ ] 네트워크 대역폭 사용량
- [ ] 오류율 및 응답 시간
- [ ] 디스크 사용량

정기 유지보수

- [ ] 로그 로테이션 및 정리
- [ ] 성능 메트릭 분석
- [ ] 보안 업데이트 적용
- [ ] 백업 데이터 검증
- [ ] 장애 복구 테스트

트러블슈팅

일반적인 문제들

1. **높은 메모리 사용량**

- 원인: 너무 많은 동시 연결
- 해결: `max_reservations`, `max_circuits` 조정

2. **연결 실패**

- 원인: 방화벽 또는 NAT 설정
- 해결: 포트 포워딩 및 방화벽 규칙 확인

3. **성능 저하**

- 원인: Rate limiting 설정이 너무 엄격
- 해결: Rate limit 값 조정

로그 분석

```
# 오류 로그 확인
grep "ERROR"  /var/log/relay-server.log

# 연결 통계 확인
grep "Circuit established"  /var/log/relay-server.log | wc -l

# 메모리 사용량 모니터링
ps aux | grep relay-server
```

이 가이드를 통해 안정적이고 확장 가능한 libp2p relay 서버를 구축하고 운영할 수 있습니다.